



LATER GATOR

DESIGN AND PLAN PRESENTATION

PREPARED FOR

CEN3031 – Introduction to Software Engineering

FALL 2025 | University of Florida

Professor Lisha Zhou

CARRIE RUBLE

TEAM LATER GATOR:

Danielle Foege

Madison Holt

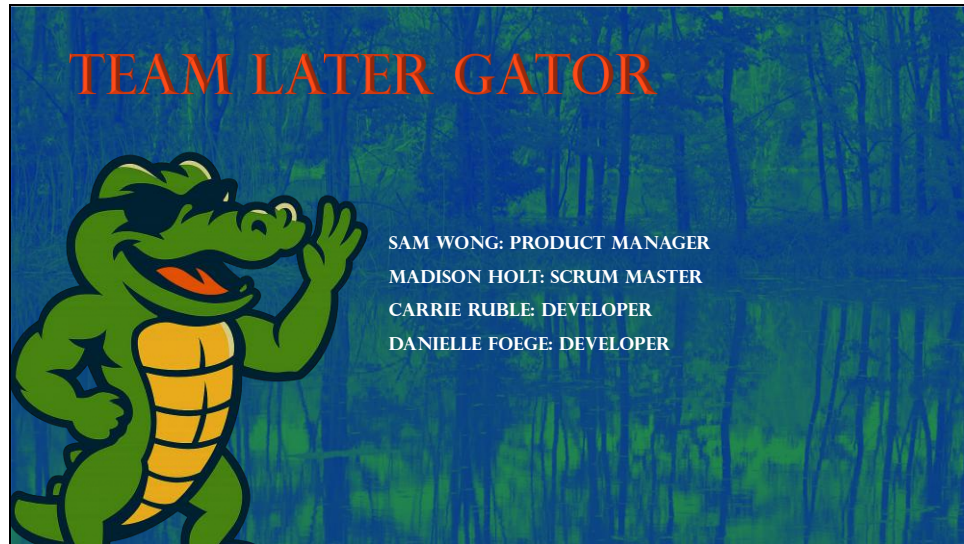
Carrie Ruble

Samantha Wong

Presentation Overview

The following pages provide the slides and corresponding speaker notes for Team Later Gator's *Design and Plan Presentation*. These materials outline our project concept, user stories, sprint planning, software architecture plan, and tools.

A recording of the full presentation can be viewed at the following unlisted YouTube link:
<https://youtu.be/EeSqSf24r6g>.



SAM:

"We are **Team Later Gator**, and today we're presenting our progress on the project. --

I'm Sam – *Project Manager*, responsible for helping define the product vision, prioritize product backlog features, and ensure the team develops feature based on the user's needs. "

OTHERS:

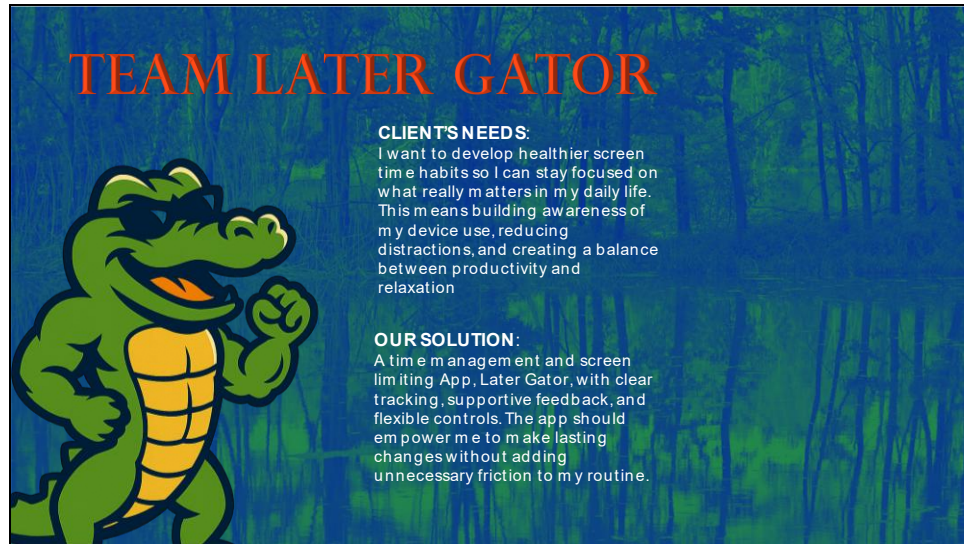
I'm **Madison** – *Scrum Master*, responsible for maintaining our Jira and ensuring our sprints go according to plan

I'm **Danielle** – *Developer*, responsible for ... Ensuring correct coding environment and tools and that product development best practices are followed.

I'm **Carrie** – *Developer*, responsible for ... I work with Danielle on keeping the team well resourced for our product development.

SAM: "Now that we've introduced our team, I'll walk us through the problem our app addresses, the proposed solution, and how our key features bring that idea to life.

[CLICK]



SAM:

The problem our project is tackling is in developing healthier screen time habits, build awareness of device use, reduce distractions, and create balance between productivity and relaxation

[CLICK]

SAM: Our solution is to create a time management and screen limiting app. That provides clear tracking and feed back to users so they can improve their habits as well as having flexible controls so that the app can be personalized to their needs.

[NEXT SLIDE]

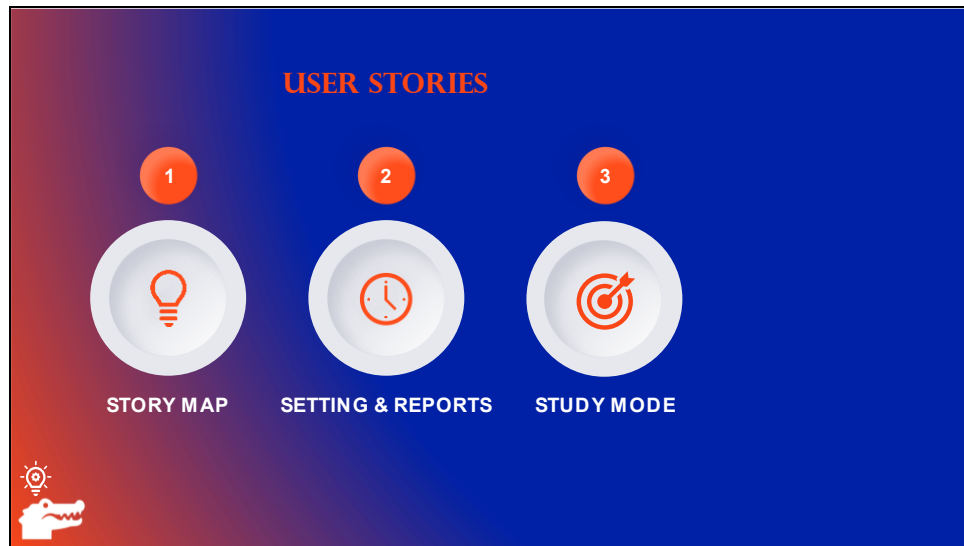
Slide 3



SAM:

Ok, Let's get into it!

[CLICK 'IDEA' GATOR ICON]

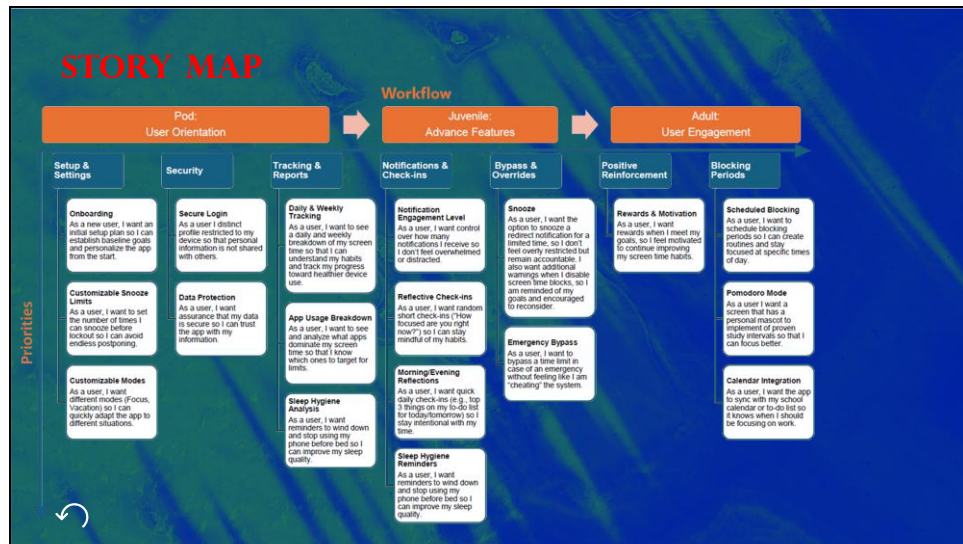


SAM:

User stories: to make sure every feature in Later Gator is designed around what the user needs. They help us break down goals, organize team discussions, and define clear acceptance criteria

[NEXT]

Slide 5



SAM:

All of our user story are shown here in the User Story Map. They're organized into user story categories. The workflow of these categories, shown horizontally. Within each step, the stories are also arranged by priority shown vertically. Some examples can be seen in the user story category of settings and report

[Next]

SETTINGS & REPORTS

Settings:

- "As a new user, I want establish baseline goals and personalize the app from the start."

Conversation:

- How should users establish a baseline?
- What features should be personalized

Confirmation:

- User sets personal goals during onboarding which is translated to setting

Reports:

- "As a user, I want to see a daily and weekly breakdown of my screen time so that I can understand my habits and track my progress toward healthier device usage"

Conversation

- What should be shown in tracking logs?
- How frequently should user receive reports?

Confirmation

- Weekly and Daily summary reports available in the app
- Visual display along with report
- Ranking of most used app usage

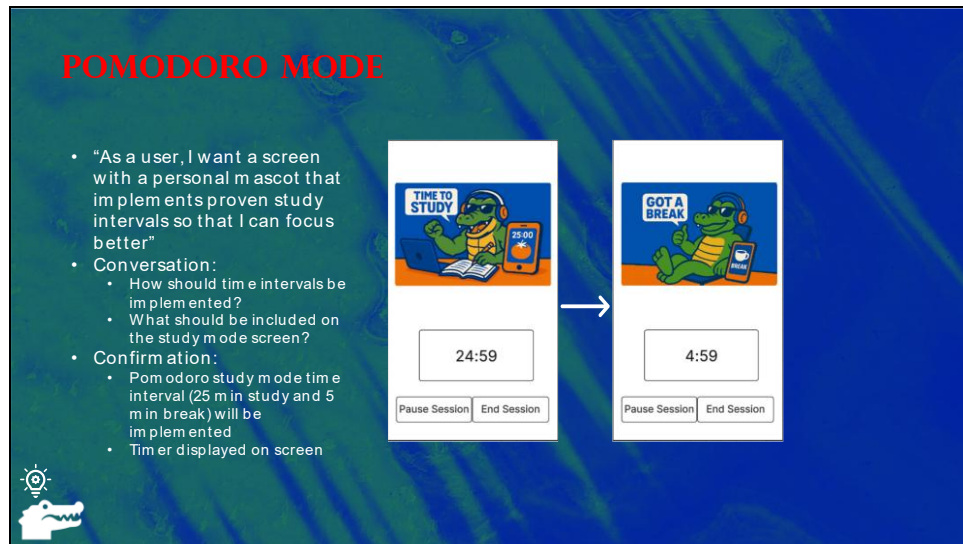
SAM:

Every story follows the three phases : Card, Conversation, and Confirmation.

The Settings story focuses on helping users establish a baseline and allowing them personalize the app. We discussed that Users should be able to set their goals/ restrictions during onboarding. Then this information being transferred to the Settings screen, where users can continue to access and edit to features such as snooze limits, app-specific time restrictions, and notification engagement levels

The Reports story focuses on helping users visualize their progress and understand their screen time habits. We discussed what information should appear in tracking logs, such as app usage, time spent in study/focus mode, and overall progress toward goals , as well as how frequently users should receive their reports. We decided that reports should be available both daily and weekly users can view daily and weekly summaries, with visual charts and rankings of their most used apps

{Next}



SAM:

The Pomodoro Study story focuses on helping users improve study habits using timed study interval. We discussed what method that time interval should be implemented and what features should be placed on the screen. We decided that the story would be confirmed if we implemented the pomodoro study method of 25 mins study and a 5 min break. There would be two screens to structure the study sessions along with a time on each screen. Users would be able to set up the number of sessions in the settings.

Now on to Danielle....



DANIELLE:

Sam walked through how the User stories gave us the opportunity to think about the features of the app,

1. [Click Creation]

Next comes breaking up those features and creating individual task cards in Jira

2. [Click Poker]

Once task cards were created, the team's next step was to point the tasks

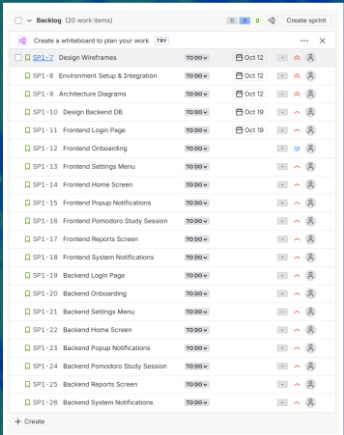
3. [Click Grooming]

After creating the cards and pointing them, it came time to perform backlog grooming

CREATING THE CARDS

Product deliverables and user stories are broken down into clear, manageable tasks.

In card creation, the focus is on creating actionable tasks that result in a minimal viable product (MVP).



Task	Estimate	Due Date	Status
SP1-7 Design Wireframes	10000	Oct 12	In Progress
SP1-8 Environment Setup & Integration	10000	Oct 12	Not Started
SP1-9 Architecture Diagrams	10000	Oct 12	Not Started
SP1-10 Design Backend DB	10000	Oct 19	Not Started
SP1-11 Frontend Login Page	10000	Oct 19	Not Started
SP1-12 Frontend Onboarding	10000		Not Started
SP1-13 Frontend Settings Menu	10000		Not Started
SP1-14 Frontend Home Screen	10000		Not Started
SP1-15 Frontend Popup Notifications	10000		Not Started
SP1-16 Frontend Pomodoro Study Session	10000		Not Started
SP1-17 Frontend Reports Screen	10000		Not Started
SP1-18 Frontend System Notifications	10000		Not Started
SP1-19 Backend Login Page	10000		Not Started
SP1-20 Backend Onboarding	10000		Not Started
SP1-21 Backend Settings Menu	10000		Not Started
SP1-22 Backend Home Screen	10000		Not Started
SP1-23 Backend Popup Notifications	10000		Not Started
SP1-24 Backend Pomodoro Study Session	10000		Not Started
SP1-25 Backend Reports Screen	10000		Not Started
SP1-26 Backend System Notifications	10000		Not Started

DANIELLE:

Creation: Broke down the project related tasks (including creating project deliverables and user stories). It's important to break tasks down so that we can manage our time and stay on schedule.

When breaking features/deliverables down, we prioritized clear and manageable tasks, considering the scope of our project and the goal of creating a minimal viable product (MVP).

As tasks were created, we also considered their priority (for example, environment setup must be done before a home screen can be built).

As seen in the image of our backlog, after breaking down our project components, creating wireframes, and developing architecture diagrams, main pages of the app were broken into frontend and backend tasks. This breakdown gives members a clear division of work and the team the ability to work on separate tasks without interfering with another's work.

[NEXT]

PLANNING POKER

Planning Poker gives each member the chance to point tasks and then discuss differences amongst the team. The process is repeated for each task, as many times as needed to reach a consensus.

The left screenshot, titled "SP1-8 Environment Setup & Integration", shows a voting interface with a bar chart. It displays individual votes of 3 and 5, with an average of 3.5 and a final result of 3. The right screenshot, titled "SP1-14 Frontend Home Screen", shows a similar voting interface with individual votes of 3 and 3, an average of 3, and a final result of 3. Both screens include a "Voting 1/1" indicator and a "Result" section.

DANIELLE:

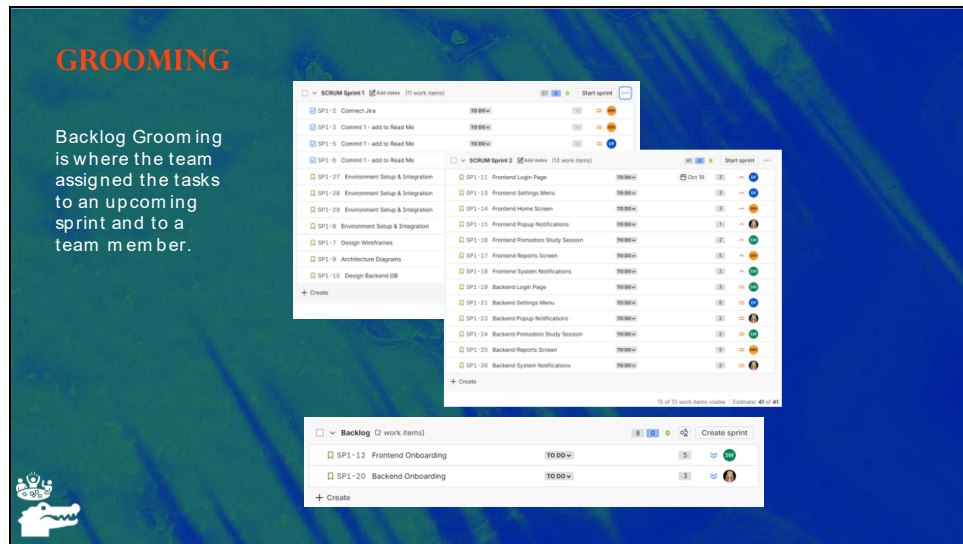
Poker: The team used an online Planning Poker app, integrated within Jira to point tasks.

If there were differences in point value estimations between team members, we discussed our thoughts and came to a consensus regarding the point value of the task. This helped us learn more about team member's experiences, confidence, and concerns pertaining to different aspects of the project. This discussion also provided an opportunity to clarify the scope and goal of the task, making sure all members were on the same page and confident in the team's ability to accomplish the task within the given point value.

As seen on the screen, we were all in agreement that developing the Frontend Home screen should take about 3 hours. However, for the Environment Setup there were differences in estimates that ultimately boiled down to varying levels of experience working with development environments.

[NEXT]

Slide 11



DANIELLE:

Grooming: Backlog grooming is when we went through and divided the tasks based on our capacity and the priority of the task. In a sprint, each member is expected to work a set number of hours, so, with a set team size, that results in a sprint capacity. These guidelines help us to commit to tasks that can realistically be accomplished within a given time frame.

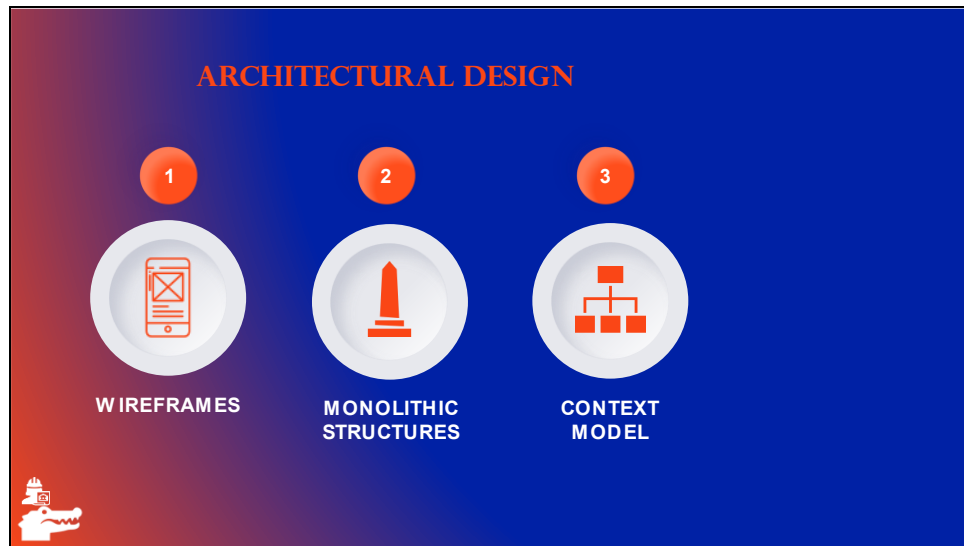
We considered team member strengths/interests and assigned tasks accordingly (encouraging pair programming opportunities to help improve the quality of our product and facilitate knowledge sharing within the team).

Sprint 1 Goal: to break down the product's architecture and create a clear vision before getting into the development. For team members that meant setting up their environment, designing wireframes, and creating architecture diagrams

Sprint 2 Goal: to develop the app features and create a prototype deliverable.

Backlog: Additional features

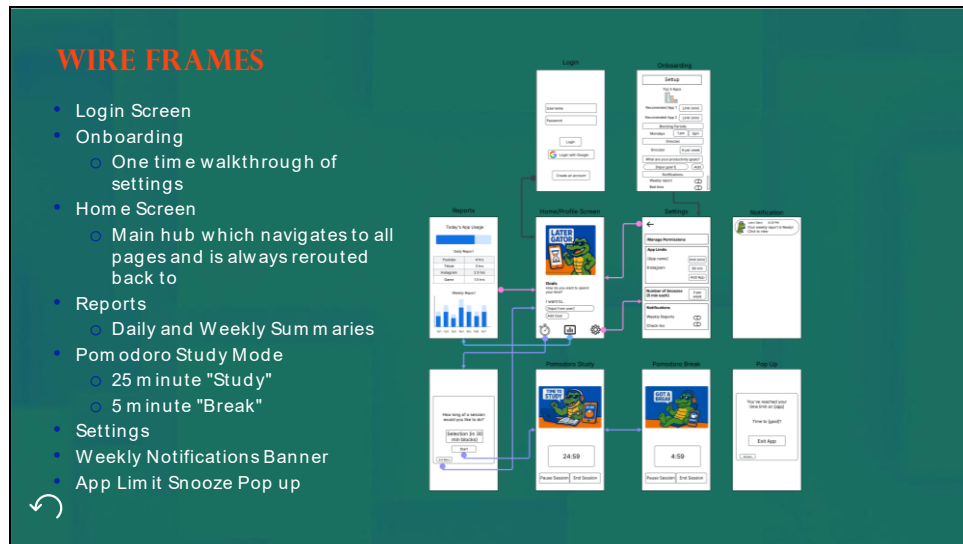
[Click Gator] Now over to Madison for a look into the architecture



MADISON:

WireFrames: Thanks Danielle, [NEXT]

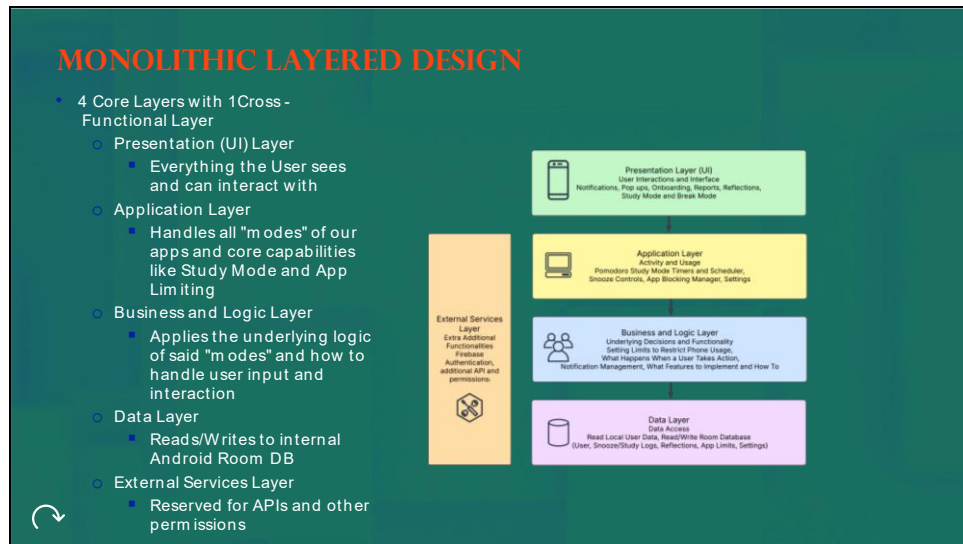
Monolithic Structures:



MADISON:

- At first we struggled to come together on a architecture design framework. Instead of staying stuck designing things we couldn't quite grasp we pivoted to something we could more rapidly develop.
- To wrap our minds around the necessary elements that would need to be made for our first prototype, we started by creating wireframes of all the main "modes" of our app
- We thought through each of the "modes" necessary for our MVP and created a wireframe for each screen and then connected them by how they might interact with each other and control navigation within the app
- Doing this allowed us to freely and rapidly prototype without focusing too much on the much more structured elements of our other architecture diagrams

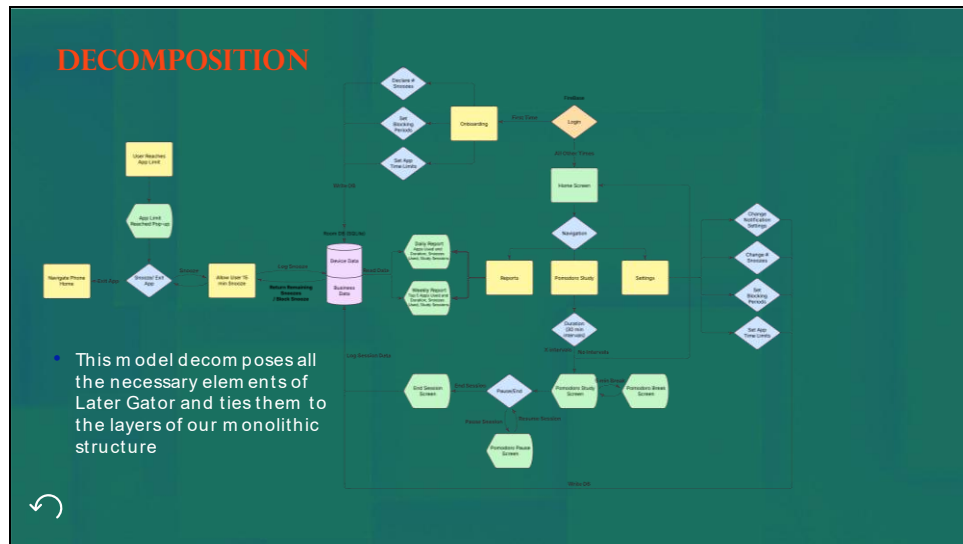
[NEXT]



MADISON:

For our Architecture design we chose a monolithic layered design with 4 core layers and one cross cutting layer. As we are developing a single mobile app for our MVP, the monolithic structure made the most sense as it simplified our pipeline for development and testing. The layers of the model also defined clear structure and modularity.

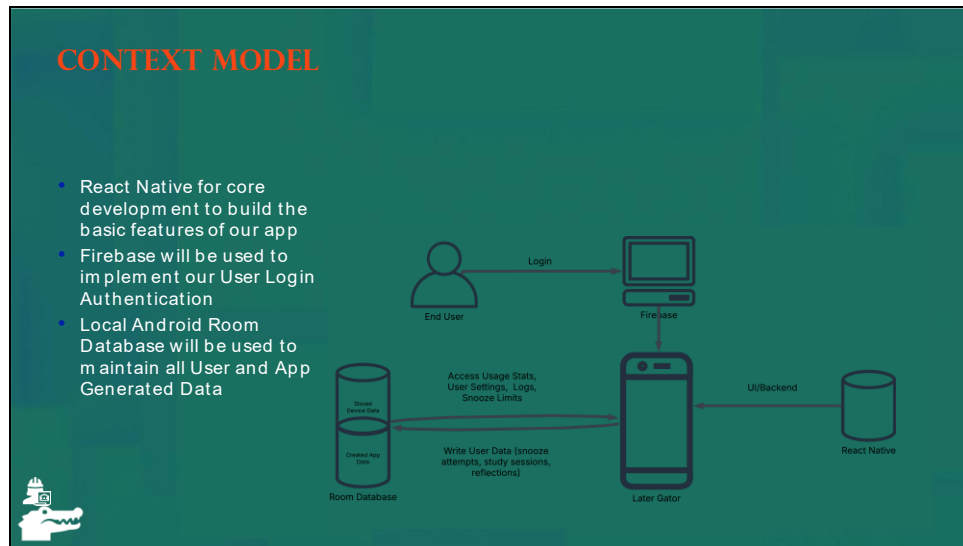
[NEXT]



MADISON:

From here we further decomposed our MVP and tied our leveled steps to the layers of our architecture model. This process helped us better understand the core functionalities and guiding principles of our MVP as well as internal logic and the flow of our App. This visual also helped us realize how the layers themselves governed our design.

[NEXT]



MADISON:

With our system decomposed and core framework we were able to create our System Context Model. Our Core development will be through React Native. Firebase will handle User login and authentication. and all data will be handled through Android's Internal Room Database. Carrie will delve more into our chosen technologies

[NEXT]



CARRIE:

Next, I'll go over the tools and technologies our team is using to plan, build, test, and eventually deploy the Later Gator app.

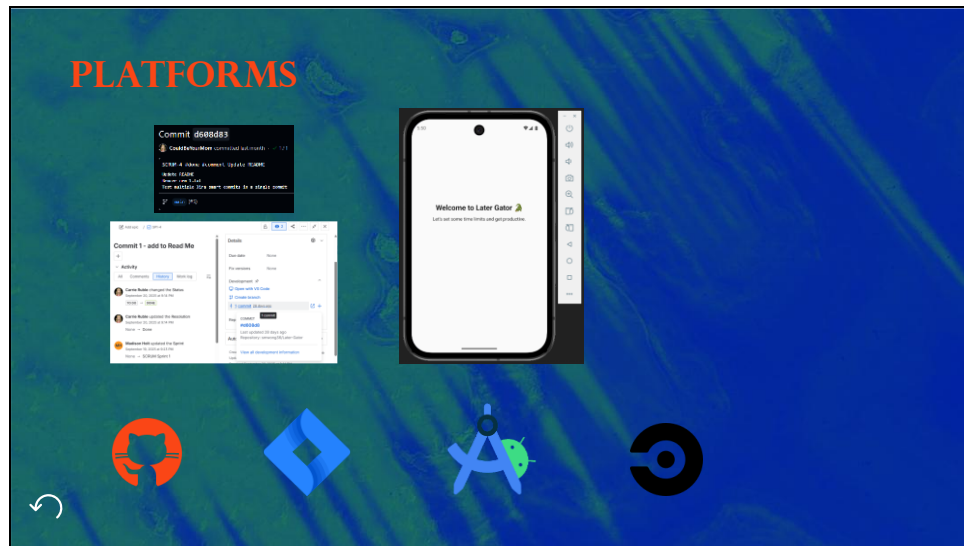
[CLICK PLATFORMS]

Now that we've covered where our work lives and how we track progress, let's look at the frameworks and libraries that will actually power our app.

[CLICK FRAMEWORKS & LIBRARIES]

Those frameworks rely on specific programming languages, so next we'll look at the main ones our team will be using.

[CLICK LANGUAGES]



CARRIE:

"Our development workflow connects four main platforms.

[CLICK]

GitHub is where we store, manage, and share the code for our project. It provides access control, and bug tracking, as well as being the foundation for continuous integration...

[CLICK]

Jira: is a project management tool for tracking, managing, and reporting on tasks and issues. We're utilizing it for agile methodologies like Scrum and Kanban.

Danielle showed you a few screenshots from our Jira Board.

Labeling each task in Jira gives us the ability to use "Smart Commits"

Here you can see a commit, as it appears in GitHub, and the same commit was able to transition a task's status from TO DO to DONE.

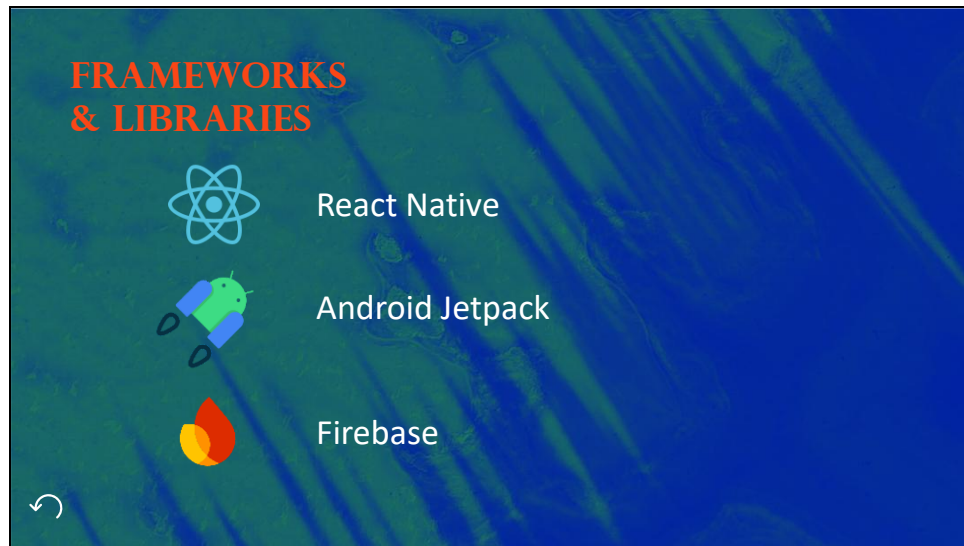
[CLICK]

Android Studio: is an IDE that provides Android Virtual Devices (AVD), or emulators like the one seen here, and a stable preconfigured project template for our app that we will build from once we start adding our features.

[CLICK]

We will be using **CircleCI** for CI/CD to automate our testing and builds once development begins."

[BACK BUTTON]



CARRIE:

"Our project uses a mix of front-end and back-end frameworks that make development faster and more reliable.

[CLICK]

On the front end, **React Native** allows us to share one codebase across Android devices while still accessing native features.

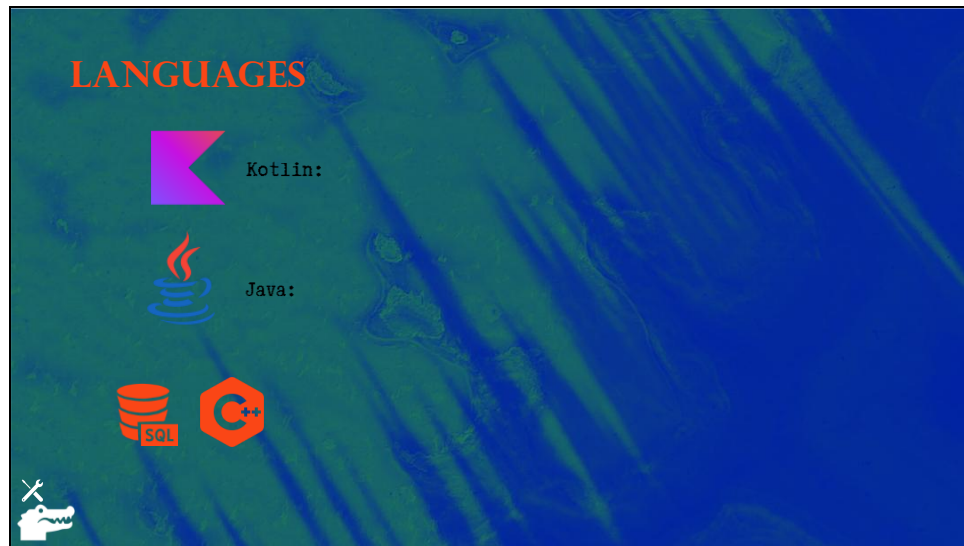
[CLICK]

Android Jetpack libraries, like Google's Maven repository and Room, provide pre-built components for database management, navigation, and lifecycle handling...

[CLICK]

We also plan to integrate **Firebase** for authentication and cloud-based storage as the project expands beyond a single-device prototype.

[BACK BUTTON WHILE SAYING THE FOLLOWING:] Altogether, these frameworks let us focus on app logic instead of reinventing the wheel for every feature."



CARRIE:

[CLICK]

Our main programming language is **Kotlin**, which runs on the same virtual machine as **Java** but allows for cleaner, safer, and more concise code.

[CLICK]

Kotlin is to Java what Python is to C, it builds on the same foundation, but with modern features like null safety, coroutines, and built-in type inference that help us write fewer bugs with less boilerplate.

[CLICK]

We will also use **SQL** for local data management through SQLite, and we may include **C++** for any performance-intensive features.

These platforms, frameworks, and languages create the foundation for everything we'll build moving forward. With our environment set up, we're ready to start implementing features.

[CLICK GATOR TOOLS ICON] And I'll pass it back to Sam to talk about some issues we've encountered and our next steps.



Sam:

Issues we've overcome:

- Madison already described the issue we encountered when facing decomposition, and how we overcame that.
- scope creep – During our user story session we came up with many creative features such as a gamification system. However, upon breaking down task for our product back log we realize that gamification features would take a lot of time and research to implement, beyond our workload capacity. As result they were deprioritized in this prototype.

Next Steps

- Ensure every team member's development environment is fully configured and tested.
- Verify everyone can build and run the app locally using the shared GitHub repository.
- Begin the next sprint focused on implementing core functionality — starting with user settings, reports, and notification features.

REFERENCES

- All icons used in this presentation are from <https://icons8.com/icons/>, and reproduced or modified from work created and shared by each respective entity and used according to terms described in the [Creative Commons](#) 3.0 Attribution License.
- Later-Gator mascot art created with ChatGPT (OpenAI).